

Prof. Dr. Stefan Leutenegger

Teaching Assistants: Shi Chen, Cédric Le Gentil, Alex Brandes, and Yannick Burkhardt

Solutions to Exercise 9: Probabilistic Occupancy Mapping

Session date: 05/05/2026

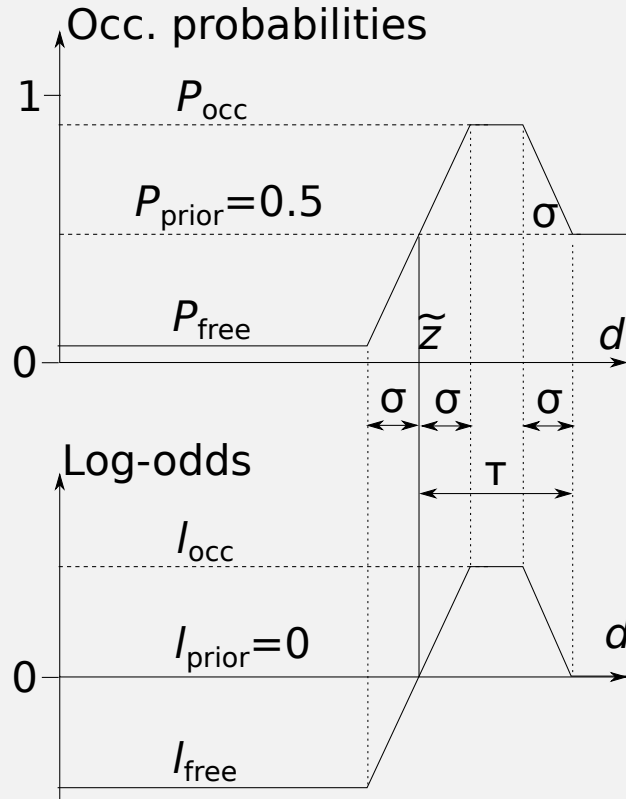
Task 1: 2D Occupancy Mapping of a Diff-Drive Robot (Python)

Study the implementation of the simulator in `E9_diff_drive_occ.py`. You can steer the robot using the arrow keys. The robot is equipped with N laser range sensors, centered at the robot's body frame $\mathcal{F}_B$ origin and pointing into different directions in angular increments of $\Delta\alpha$. Each sensor with index $i = 0, \dots, N - 1$ is oriented at an angle α_i relative to the robot's x axis: $\alpha_i = -\Delta\alpha(N - 1)/2 + i\Delta\alpha$. The simulator generates measurements $\tilde{z}_i$ containing Gaussian noise and a uniform outlier component in the interval $[0, d_{\max}]$. To flag an invalid measurement (e.g., due to specular reflection or excessive range), $\tilde{z}_i = d_{\max}$ is returned.

Since we are focusing on mapping here, we assume that poses are given; we use the simulated ground-truth poses $\mathbf{x} = [x_R, y_R, \theta_R]^T$, where (x_R, y_R) is the 2D position in the world frame $\mathcal{F}_W$ and θ_R is the heading angle. You are now meant to implement the Log-Odds 2D occupancy mapping algorithm from the lecture. Note that the visualisation of the Log-Odds occupancy grid is already implemented.

- i) Propose an inverse sensor model and draw it as a graph with occupancy probability $P(\cdot)$ on the y -axis and distance from the sensor on the x -axis (with the necessary parameters labelled). Also draw it in the form we will later need, where the y -axis shows the Log-Odds value $l(\cdot)$. Make sure to accommodate the fact that there are outliers, possible violations of the static-world assumption, as well as distance measurement noise.

In the following, we schematically depict the chosen inverse sensor model in occupancy probability and log-odds form. The parameters are: measurement $\tilde{z}$, uncertainty parameter σ , and wall thickness τ . Furthermore, $P_{\text{occ}} < 1$ and $P_{\text{free}} > 0$. We assume $P_{\text{prior}} = 0.5$, which is a convenient and commonly used choice.



Note that, for ease of implementation, we use a linear slope in the log-odds domain (consequently, the slope in the occupancy probability domain is not linear).

- ii) (Python) Now implement the update function `OccupancyMap.update(x, z)`, where z contains all measurements $\tilde{z}_i$ and $\mathbf{x}$ is the full state vector. Make sure to saturate the Log-Odds values at some minimum and maximum.

To simplify things, assume perfect laser measurement beams, i.e. no need to account for a cone-like measurement. To further simplify, assume that as soon as a cell is intersected by a ray, the respective Log-Odds occupancy is updated according to the inverse sensor model above, irrespective of how close to the cell centre the ray passes.

Note: you will need to consider efficiency to make the simulation run at interactive rates. Make sure to pass the 2D array only once (not N times) per update step. Also try to make use of an early-abort mechanism when a cell is beyond sensor range or outside the angular frustum of all laser beams.

See `E9_diff_drive_occ_sol.py`. Regarding efficiency: since there is no beam opening angle to consider, the simplest approach is to iterate over all cells and check the four corners of each cell with respect to which side of each beam they lie on (using the 2D cross product). If the signs are mixed, the cell is intersected by that beam and the log-odds update is applied.

Task 2: 2D LiDAR-Based Self-Localization using an ESDF Map (Python)

The robot navigates the ETH-logo maze, whose walls are encoded in a pre-built truncated *Euclidean Signed Distance Function* (ESDF) map $\Phi: \mathbb{R}^2 \rightarrow [-\tau, \tau]$. At any world point $\mathbf{p}$, $\Phi(\mathbf{p})$ equals the signed Euclidean distance to the nearest wall surface, clamped to $\pm\tau$: positive inside navigable corridors, negative inside solid walls, and exactly zero on a wall surface. The map also provides the spatial gradient $\nabla\Phi(\mathbf{p})$, accessible via `ESDFMap.query(p)`.

The robot carries a full-circle LiDAR with N beams. Beam i ($i = 0, \dots, N-1$) is oriented at angle $\alpha_i = 2\pi i/N$ in the body frame $\mathcal{F}_B$ and returns range z_i . As in Task 1, measurements include Gaussian noise and an outlier component; $\tilde{z}_i = d_{\max}$ flags an invalid beam. Note that, unlike a real rotating LiDAR, all beams are modelled as a synchronised snapshot captured simultaneously, and the pose correction is applied every few frames to simplify the problem. The robot state is $\mathbf{x} = [x_R, y_R, \theta_R]^\top$, where (x_R, y_R) is the 2D position in the world frame $\mathcal{F}_W$ and θ_R is the heading angle. For a given pose $\mathbf{x}$, the world-frame endpoint of beam i is

$$\mathbf{p}_i(\mathbf{x}) = \begin{bmatrix} x_R + z_i \cos(\theta_R + \alpha_i) \\ y_R + z_i \sin(\theta_R + \alpha_i) \end{bmatrix}.$$

- i) (Python) Open `E9_diff_drive_esdf.py` and run the simulation. Try to navigate the robot to the goal using the arrow keys. The displayed blue robot shows the estimated pose, which relies on odometry integration only (no LiDAR correction yet). What do you observe?

Hint: Press `D` to toggle debug mode.

Without LiDAR correction the estimated pose relies on clean odometry integration while the real pose accumulates noise at every step. The discrepancy grows until the real pose (gray, visible in debug mode) enters a wall and the robot crashes, even though the displayed blue estimate appears to follow the intended path correctly.

- ii) Define the error $e_i(\mathbf{x})$ for beam i with valid range $z_i < d_{\max}$, and write the cost function $c(\mathbf{x})$.

Hint: Recall that $\tilde{z}_i = d_{\max}$ flags an invalid measurement.

We define the error as $e_i(\mathbf{x}) = \Phi(\mathbf{p}_i(\mathbf{x}))$, which is zero when $\mathbf{p}_i$ lies exactly on a wall surface. The cost function is

$$c(\mathbf{x}) = \sum_i e_i(\mathbf{x})^2,$$

where the sum runs over beams with $z_i < d_{\max}$ (excluding outlier measurements). Beams whose endpoints fall in the truncated region ($|\Phi(\mathbf{p}_i)| \geq \tau$) have $\nabla\Phi = 0$ there, so they contribute zero Jacobians and do not affect the solution.

- iii) Derive the Jacobian $\mathbf{E}_i = \frac{\partial e_i}{\partial \mathbf{x}} \in \mathbb{R}^{1 \times 3}$.

Hint: First compute $\frac{\partial \mathbf{p}_i}{\partial \mathbf{x}} \in \mathbb{R}^{2 \times 3}$ entry by entry, then combine with $\nabla\Phi(\mathbf{p}_i)$ using the chain rule.

By the chain rule:

$$\mathbf{E}_i = \frac{\partial e_i}{\partial \mathbf{x}} = \nabla \Phi(\mathbf{p}_i)^\top \frac{\partial \mathbf{p}_i}{\partial \mathbf{x}}.$$

Computing $\frac{\partial \mathbf{p}_i}{\partial \mathbf{x}}$ entry by entry:

$$\frac{\partial \mathbf{p}_i}{\partial \mathbf{x}} = \frac{\partial}{\partial [x_R, y_R, \theta_R]} \begin{bmatrix} x_R + z_i \cos(\theta_R + \alpha_i) \\ y_R + z_i \sin(\theta_R + \alpha_i) \end{bmatrix} = \begin{bmatrix} 1 & 0 & -z_i \sin(\theta_R + \alpha_i) \\ 0 & 1 & z_i \cos(\theta_R + \alpha_i) \end{bmatrix} \in \mathbb{R}^{2 \times 3}.$$

Hence, writing $\nabla \Phi(\mathbf{p}_i) = \left[\frac{\partial \Phi}{\partial p_x}, \frac{\partial \Phi}{\partial p_y} \right]^\top$:

$$\mathbf{E}_i = \begin{bmatrix} \frac{\partial \Phi}{\partial p_x} & \frac{\partial \Phi}{\partial p_y} \end{bmatrix}_{\mathbf{p}_i} \begin{bmatrix} 1 & 0 & -z_i \sin(\theta_R + \alpha_i) \\ 0 & 1 & z_i \cos(\theta_R + \alpha_i) \end{bmatrix} \in \mathbb{R}^{1 \times 3}.$$

iv) Write down the normal equations $\mathbf{A} \Delta \mathbf{x} = \mathbf{b}$ from which we can solve for the update $\Delta \mathbf{x}$.

The Gauss-Newton normal equations are:

$$\mathbf{A} \Delta \mathbf{x} = \mathbf{b}, \quad \mathbf{A} = \sum_i \mathbf{E}_i^\top \mathbf{E}_i \in \mathbb{R}^{3 \times 3}, \quad \mathbf{b} = - \sum_i \mathbf{E}_i^\top e_i \in \mathbb{R}^3,$$

where the sums run over all valid beams (those with $z_i < d_{\max}$). Once $\Delta \mathbf{x}$ is found by solving this system, the state is updated as $\mathbf{x} \leftarrow \mathbf{x} + \Delta \mathbf{x}$ and the process is iterated until convergence.

v) (Python) Using the Gauss-Newton optimisation scheme, implement `PoseEstimator.estimate(z, x0)` in `E9_diff_drive_esdf.py`. Validate your implementation by navigating the robot successfully to the goal.

Hint: You may reuse the Gauss-Newton solver you implemented in Exercise 6, or simply use `numpy.linalg.solve()`.

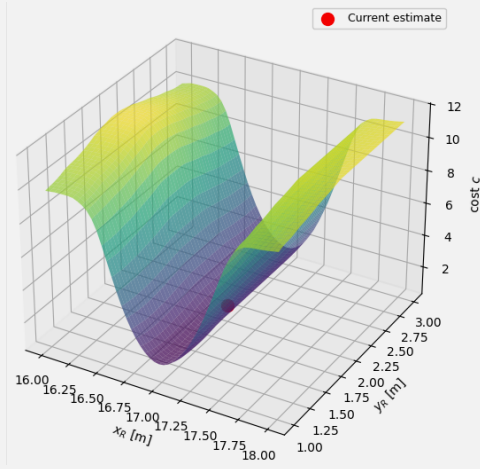
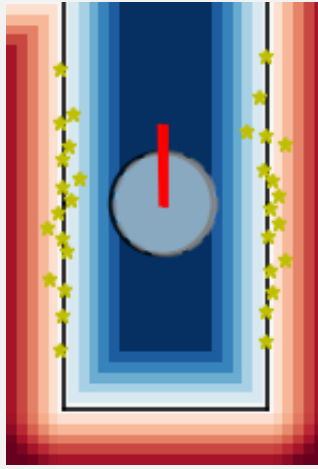
See `E9_diff_drive_esdf_sol.py`.

vi) (Python) Dial down the maximum LiDAR range `MAX_RANGE` to an extremely low 2.0 in `E9_diff_drive_esdf.py` and try the navigation again. What happens to the matrix $\mathbf{A}$ in the normal equations and why does it happen? How could it be solved?

Consider a long corridor in the ETH-logo maze with walls running along the y -direction. With $\text{MAX_RANGE} = 2.5$ m, the beams are too short to reach the ends of the corridor. Only the two side walls (perpendicular to the x -axis) are hit, so all valid endpoints $\mathbf{p}_i$ lie on surfaces whose normal is $\hat{\mathbf{n}} = [\pm 1, 0]^\top$. Since $\nabla \Phi(\mathbf{p}_i) \approx \hat{\mathbf{n}}$, every valid Jacobian takes the form

$$\mathbf{E}_i \approx \begin{bmatrix} \pm 1 & 0 & \mp z_i \sin(\theta_R + \alpha_i) \end{bmatrix}.$$

The second column is always zero, so $\mathbf{A} = \sum_i \mathbf{E}_i^\top \mathbf{E}_i$ is rank-deficient ($\text{rank}(\mathbf{A}) = 2$) and $\mathbf{A} \Delta \mathbf{x} = \mathbf{b}$ cannot be solved uniquely. Intuitively, observing only the two side walls tells us nothing about where along the corridor the robot is. Note that the x -position and heading θ_R are still well-constrained. The LiDAR scan snapshot and the resulting cost landscape $c(x_R, y_R)$ (θ_R omitted for simplicity) are shown below: the LiDAR hits the two side walls only, and the cost landscape forms a flat ridge along y_R , confirming that the optimiser has no gradient signal to recover the along-corridor position.



How could it be solved? One practical fix is to skip the update whenever $\mathbf{A}$ is detected to be rank-deficient. A more principled approach is *Levenberg–Marquardt*, of which two common variants exist:

- **Identity damping**

$$(\mathbf{A} + \lambda \mathbf{I}) \Delta \mathbf{x} = \mathbf{b}.$$

- **Pro:** Guarantees full rank and numerical stability.
- **Con:** The same damping to all directions, unnecessarily reducing the step size even in flat directions.

- **Diagonal damping**

$$(\mathbf{A} + \lambda \text{diag}(\mathbf{A})) \Delta \mathbf{x} = \mathbf{b}.$$

- **Pro:** Damping proportional to the curvature, keeping the step size large in flat directions (Recall that $\mathbf{A}$ approximates the Hessian of the cost).
- **Con:** If a diagonal entry is exactly zero (as happens here), this variant fails to fix the rank deficiency. Near-zero entries can additionally cause numerically large, unstable steps.

Both hyperparameters `LAMBDA_LM_DIAG` and `LAMBDA_LM_I` are available in `E9_diff_drive_esdf_sol.py`; try adjusting each and observe how it affects the update behaviour in the degenerate corridor. See `E9_diff_drive_esdf_sol.py` for the reference implementation.